

SMART CAMPUS UPB

Plateforme IoT sécurisée de supervision des infrastructures
de l'Université Polytechnique de Bingerville

Master 1 — RIST / BD-GL

Mini-projet Hackathon — IoT Security × Développement IoT

Équipe :

ANGO CHIAYE DESIREE VICTOIRE

TOURE YEGUE ANNE-LORENE

SILUE ELIE

1. Contexte métier et problématique

L'Université Polytechnique de Bingerville (UPB) ne dispose pas d'un système centralisé permettant de surveiller en temps réel les salles de cours, les bâtiments, les équipements et la consommation énergétique du campus. Cette absence de visibilité entraîne un gaspillage énergétique (climatisation ou éclairage laissés actifs dans des salles vides) et complique la gestion des infrastructures.

Ce projet propose une plateforme IoT de supervision du campus, capable de collecter les données de capteurs simulés (température, humidité, présence, consommation électrique, état des portes, qualité de l'air, détection de fumée), de détecter des anomalies, et de fournir un tableau de bord sécurisé aux administrateurs.

Le projet répond à un double objectif pédagogique : d'une part, concevoir une architecture IoT distribuée avec haute disponibilité (Redis Sentinel, API multi-instances derrière un load balancer Nginx) ; d'autre part, démontrer la sécurisation d'un système IoT de bout en bout, à travers un cycle attaque → preuve → contre-mesure → validation.

2. Présentation de l'équipe et répartition des rôles

Le projet a été réalisé par une équipe de 3 étudiants, avec une répartition des rôles alignée sur les spécialités du Master :

Rôle	Responsabilités	Livrables produits
Sécurité & Architecture réseau	Déploiement du broker MQTT, conduite des scénarios d'attaque, sécurisation TLS, analyse STRIDE	Scripts d'attaque, captures de preuves, matrice STRIDE
Backend, BD & Cache	API REST (FastAPI, 3 instances), bases de données (InfluxDB/MongoDB), cache Redis (Sentinel), authentification JWT	Code API, architecture Redis HA, tests
Frontend & Observabilité	Dashboard React (6 vues), intégration Grafana/Prometheus, docker-compose.yml	Code React, stack d'observabilité

3. Architecture générale

Capteurs IoT (simulés)

- ↓ MQTT (Mosquitto, TLS activé sur le port 8883)
- ↓ [pipeline backend — voir section Backend]
- ↓ API FastAPI (3 instances) — Nginx (load balancer)
- ↓ Dashboard React
 - | — Grafana / Prometheus (observabilité)

Le dépôt du projet est structuré en trois dossiers correspondant aux trois rôles : securite/, backend/, frontend/.

4. Partie Sécurité

4.1 Architecture de sécurité mise en place

Un broker MQTT (Eclipse Mosquitto) a été déployé via Docker pour recevoir les données publiées par un capteur simulé en Python (bibliothèque paho-mqtt), publiant toutes les 5 secondes des données de température, présence et identifiant de salle au format JSON sur le topic campus/<salle>/data.

Dans une première phase, le broker a été délibérément laissé sans chiffrement (port 1883) afin de démontrer les vulnérabilités d'une architecture IoT non sécurisée. Dans une seconde phase, une infrastructure TLS a été mise en place (autorité de certification, certificat serveur, port 8883) pour sécuriser les communications.

4.2 Scénarios d'attaque et preuves

A1 — Sniffing MQTT

Vecteur : Capture du trafic réseau MQTT en clair avec Wireshark, filtre mqtt.

Résultat : Les messages publiés par le capteur (température, présence, identifiant de salle) sont visibles en clair dans le flux réseau et dans le payload MQTT, sans aucune protection.

Preuve : Capture Wireshark montrant le topic campus/A101/data et le payload JSON en texte lisible.

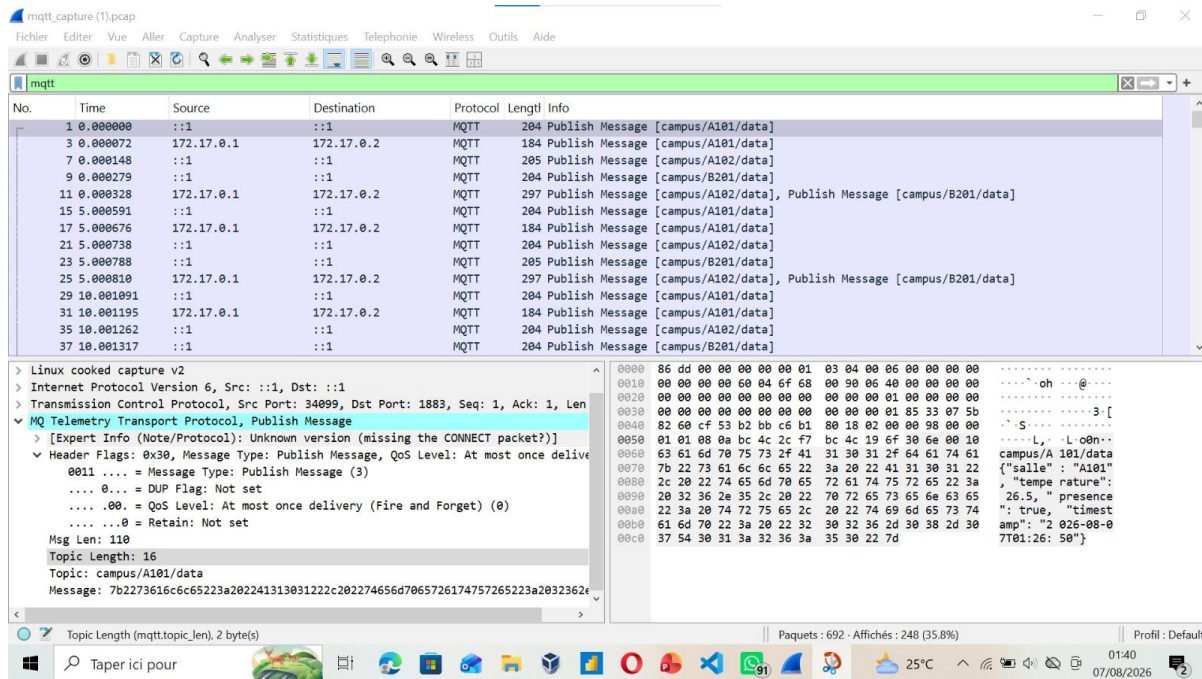


Figure 1 — Capture Wireshark du trafic MQTT en clair (port 1883) : payload JSON lisible "salle", "temperature", "presence"

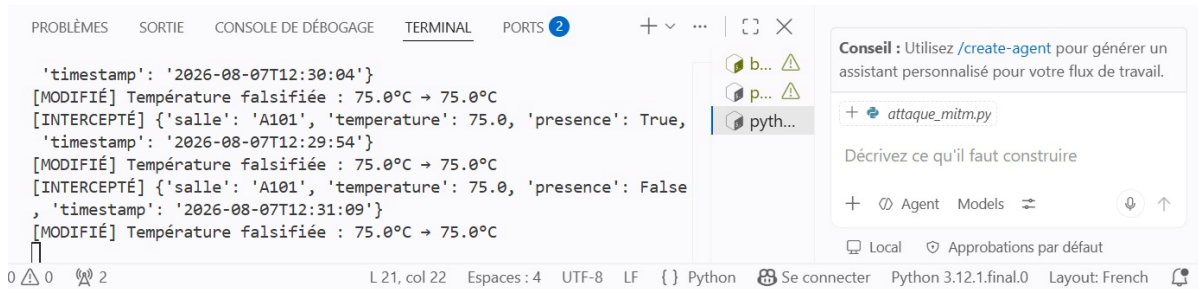
Impact : Un attaquant positionné sur le réseau peut intercepter passivement toutes les données du système sans laisser de trace, compromettant la confidentialité des données de supervision du campus.

A2 — MITM (Man-in-the-Middle)

Vecteur : Script Python interceptant les messages publiés sur le topic campus/A101/data, modifiant la valeur de température avant republication sur le même topic.

Résultat : Une température de 22-28°C (valeur légitime) a été systématiquement transformée en 75.0°C avant d'être republiée, simulant une donnée capable de déclencher une fausse alerte incendie côté dashboard.

Preuve : Capture du terminal montrant en boucle [INTERCEPTÉ] (valeur réelle) suivi de [MODIFIÉ] (valeur falsifiée à 75.0°C).



```
'timestamp': '2026-08-07T12:30:04'}
[MODIFIÉ] Température falsifiée : 75.0°C → 75.0°C
[INTERCEPTÉ] {'salle': 'A101', 'temperature': 75.0, 'presence': True,
'timestamp': '2026-08-07T12:29:54'}
[MODIFIÉ] Température falsifiée : 75.0°C → 75.0°C
[INTERCEPTÉ] {'salle': 'A101', 'temperature': 75.0, 'presence': False
, 'timestamp': '2026-08-07T12:31:09'}
[MODIFIÉ] Température falsifiée : 75.0°C → 75.0°C
```

Figure 2 — Terminal de l'attaque MITM : interception et falsification de la température (→ 75.0°C)

Impact : Sans protection de l'intégrité des messages, un attaquant peut injecter de fausses données critiques, entraînant des décisions erronées (fausses alertes, ou à l'inverse masquage d'un incident réel).

A3 — Replay Attack

Vecteur : Capture d'un message légitime (presence: true pour la salle A101), puis rejeu de ce même message plusieurs heures après sa capture d'origine (timestamp non actualisé).

Résultat : Le message rejoué est accepté par le broker comme un message normal, faisant croire au système que la salle A101 est encore occupée alors que la donnée est obsolète.

Preuve : Capture du terminal montrant [MESSAGE CAPTURÉ] avec le timestamp d'origine, puis [REJEU] republiant ce même message.



```
@ANGOROXANE →/workspaces/smart-campus-upb (main) $ python securite/a
ttaque_replay.py
True, 'timestamp': '2026-08-07T10:15:00'}
Attente avant rejeu (simule un délai de plusieurs heures)...
[REJEU] Rejoue le message capturé, timestamp original : 2026-08-07T10
:15:00
[REJEU] Message republié → le système croit la salle A101 encore occu
pée
@ANGOROXANE →/workspaces/smart-campus-upb (main) $
```

Figure 3 — Terminal de l'attaque replay : rejeu du message capturé, le système accepte la donnée obsolète

Impact : L'absence de mécanisme anti-rejeu (nonce, fenêtre temporelle stricte) permet à un attaquant de faire perdurer artificiellement un état obsolète du système.

A4 — Attaque API

Vecteur : Une fois l'API backend disponible (FastAPI, 3 instances derrière Nginx), deux tests ont été conduits :

- Tentative de brute force sur l'endpoint POST /auth/login (5 tentatives consécutives avec mots de passe incorrects)
- Tentative d'accès à l'endpoint GET /measurements/latest sans jeton d'authentification (JWT)

Résultat :

- Les 5 tentatives de brute force ont toutes retourné un statut 401 Invalid credentials, sans aucun blocage ni code 429 Too Many Requests observé — l'API ne dispose pas (à ce stade du développement) d'une protection anti-brute-force spécifique sur la route d'authentification.
- La requête sur /measurements/latest sans jeton a été acceptée (statut 200, réponse []), révélant que cette route n'exige pas d'authentification JWT.

Preuve : Captures des réponses curl montrant les codes HTTP obtenus pour chaque test.

```

@ANGOROXANE → /workspaces/smart-campus-upb/backend (main) $ for i in 1 2 3 4 5; do
  curl -s -X POST http://localhost:8000/auth/login \ ...
  {"detail":"Invalid credentials"} [HTTP 401]
  {"detail":"Invalid credentials"} [HTTP 401]
  {"detail":"Invalid credentials"} [HTTP 401]
  {"detail":"Invalid credentials"} [HTTP 401]
  {"detail":"Invalid credentials"} [HTTP 401]
@ANGOROXANE → /workspaces/smart-campus-upb/backend (main) $
  
```

Figure 4 — 5 tentatives de connexion avec mots de passe incorrects sur /auth/login : toutes en 401, aucun blocage

```

@ANGOROXANE → /workspaces/smart-campus-upb/backend (main) $ curl -s -i http://localhost:8000/measurements/latest
Content-Type: application/json
Content-Length: 2
Connection: keep-alive
x-ratelimit-limit: 120
x-backend-instance: backend-3
[]
@ANGOROXANE → /workspaces/smart-campus-upb/backend (main) $
  
```

Figure 5 — Requête sans jeton JWT sur /measurements/latest : réponse acceptée, aucune authentification exigée

Impact : L'absence de blocage sur les tentatives de connexion expose le système à des attaques de force brute prolongées. L'absence d'authentification sur les routes de données expose les données de mesure du campus à toute personne connaissant l'URL de l'API.

4.3 Contre-mesures et validation

Sécurisation TLS du broker MQTT

Une infrastructure de certificats a été mise en place (autorité de certification auto-signée, certificat serveur) et Mosquitto a été reconfiguré pour exposer un listener TLS sur le port 8883, en plus du port 1883 non sécurisé (conservé à des fins de comparaison).

Validation : L'attaque de sniffing a été reproduite sur le port 8883. La capture Wireshark confirme l'établissement d'une session TLSv1.2 (paquet « Client Hello ») et l'illisibilité totale du contenu des messages échangés (données chiffrées), contrairement à la capture initiale sur le port 1883.

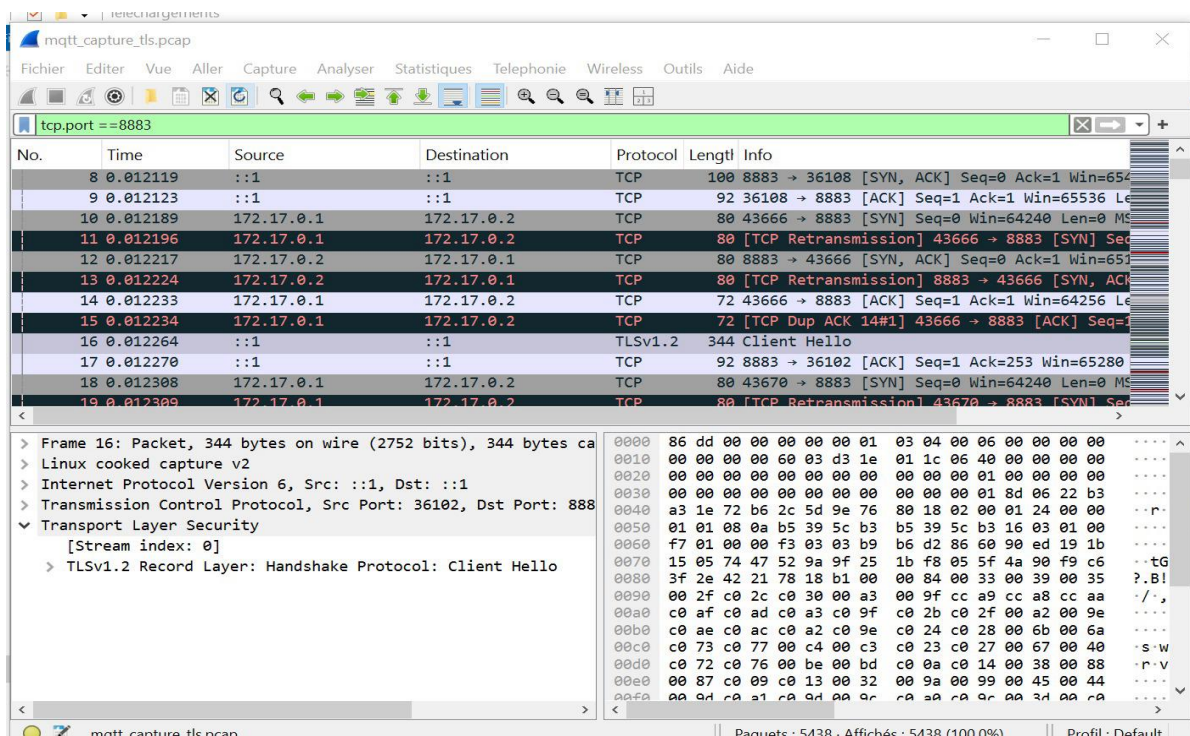


Figure 6 — Capture Wireshark après sécurisation (port 8883) : session TLSv1.2 établie, contenu chiffré et illisible

Contre-mesures recommandées (non implémentées à ce stade, pistes d'amélioration)

- Anti-brute-force sur /auth/login : limitation du nombre de tentatives par IP/compte avec verrouillage temporaire après échecs répétés
- Authentification JWT systématique sur toutes les routes exposant des données (actuellement partielle)
- Authentification mutuelle par certificat (mutual TLS) côté capteurs, pour empêcher l'usurpation d'identité (spoofing) démontrée par l'attaque replay

4.4 Analyse STRIDE

Menace	Composant	Description	Preuve / Impact	Contre-mesure
Spoofing	Broker MQTT	Un attaquant peut se faire passer pour un capteur légitime et publier de fausses données	Démontré par l'attaque replay (message rejoué avec ancien timestamp)	Authentification par certificat client (mutual TLS)
Tampering	Communication MQTT	Modification d'un message en transit	Démontré par l'attaque MITM (température 27°C falsifiée en 75°C)	TLS activé sur le broker (port 8883) — fait
Repudiation	API Backend	Absence de logs détaillés liant chaque action à un utilisateur identifié	À vérifier avec l'équipe backend	Logging des requêtes avec horodatage, IP et identifiant utilisateur
Information disclosure	Broker MQTT (avant sécurisation)	Données capteurs lisibles en clair sur le réseau	Démontré par capture Wireshark (sniffing MQTT, port 1883)	TLS activé sur le broker — fait

Menace	Composant	Description	Preuve / Impact	Contre-mesure
Denial of service	API Backend (/auth/login)	Absence de blocage après tentatives de connexion répétées	Démontré par 5 tentatives de brute force consécutives sans blocage (toutes en 401)	Renforcer le rate limiting spécifiquement sur les routes d'authentification
Elevation of privilege	API Backend (/measurements/latest)	Accès aux données de mesure sans authentification	Démontré par requête curl sans token, réponse HTTP 200	Exiger un token JWT valide sur toutes les routes de données

4.5 Conclusion de la partie sécurité

Les quatre scénarios d'attaque exigés par le sujet ont été menés avec succès sur l'infrastructure du projet, avec preuves à l'appui (captures Wireshark et captures terminal). La sécurisation TLS du canal MQTT a été mise en œuvre et validée par la reproduction de l'attaque de sniffing après sécurisation, confirmant l'efficacité de la contre-mesure. L'attaque API a permis d'identifier des axes d'amélioration concrets pour l'équipe backend (rate limiting renforcé, authentification systématique), qui font l'objet de recommandations dans l'analyse STRIDE ci-dessus.

5. Partie Backend, BD & Cache

5.1 Résumé exécutif

Le backend Smart Campus UPB a été finalisé autour d'une architecture FastAPI modulaire et dockerisée. Le travail couvre les fondations API, la persistance MongoDB et InfluxDB, la sécurité JWT/RBAC, le cache Redis, la haute disponibilité (Sentinel, Nginx), l'ingestion Node-RED/MQTT, les métriques et la documentation d'exploitation.

Domaine	Résultat livré
API	FastAPI avec Swagger/OpenAPI, healthchecks, endpoints métier et ingestion
Données	MongoDB pour entités métier, InfluxDB pour mesures temporelles
Sécurité	JWT, hash bcrypt des mots de passe, rôles user/admin
Performance	Redis cache-aside, TTL, invalidation et rate limiting partagé

Domaine	Résultat livré
Haute disponibilité	3 instances FastAPI derrière Nginx, Redis Sentinel
Observabilité	Endpoint Prometheus /metrics et readiness dependencies

5.2 Phases réalisées

- Phase 1 : fondation FastAPI, configuration, logging, health endpoints, Swagger, Dockerfile et Pytest
- Phase 2 : MongoDB avec Motor, InfluxDB officiel, modèles Pydantic, repositories, services, endpoints rooms/sensors/measurements, docker-compose initial et tests
- Phase 3 : authentification JWT, création utilisateur, login, token expiré, utilisateur courant, rôles user/admin et permissions
- Phase 4 : Redis cache-aside, TTL, comportement tolérant si Redis est indisponible, invalidation après écriture
- Phase 5 : Redis Pub/Sub pour propager les invalidations entre instances
- Phase 6 : rate limiting basé Redis, limite partagée entre instances, réponse HTTP 429
- Phase 7 : Redis Sentinel, master, replicas et configuration de découverte du master
- Phase 8 : trois instances FastAPI et Nginx load balancer
- Phase 9 : interface d'ingestion compatible Node-RED/MQTT
- Phase 10 : métriques Prometheus
- Phase 11 : tests Pytest et couverture mesurée à 82 % lors de la dernière validation complète
- Phase 12 : documentation backend complète
- Phase 13 : validation fonctionnelle partielle Docker, avec point restant sur le failover Sentinel long

5.3 Architecture finale

Le flux cible est : Client → Nginx → backend-1 / backend-2 / backend-3 → MongoDB, InfluxDB et Redis via Sentinel.

Couche	Rôle
app/api	Routes HTTP : auth, health, rooms, sensors, measurements, ingestion, metrics
app/models	Schémas Pydantic : User, Room, Sensor, Measurement, Event
app/repositories	Accès MongoDB et transformation documents → models
app/services	Logique métier : cache, catalogue, events Pub/Sub, measurements, metrics, rate limit
app/db	Clients MongoDB, InfluxDB, Redis/Sentinel
app/core	Configuration, sécurité, exceptions, logging

5.4 Fonctionnalités backend

5.4.1 MongoDB

- Collections prévues : users, rooms, sensors, events
- Index : users_email_unique, rooms_name_unique, sensors_room_name_unique, events_created_at
- Repository Mongo centralisé pour création/lecture rooms, sensors et users
- Connexion Motor asynchrone, ping readiness, fermeture propre du client au shutdown

5.4.2 InfluxDB

- Measurement : campus_measurements
- Tags : room, sensor
- Fields : temperature, presence
- Timestamp : fourni ou généré côté API
- Fonctions : écriture, latest, history avec cache latest et invalidation après écriture

5.4.3 Authentification et RBAC

- POST /auth/register crée un utilisateur user
- POST /auth/login retourne un JWT bearer avec expiration
- GET /auth/me retourne l'utilisateur courant
- Mots de passe hashés avec bcrypt ; aucun mot de passe en clair stocké
- Routes d'administration protégées par le rôle admin

5.4.4 Redis, Sentinel et Nginx

- Cache-aside sur rooms, sensors et latest measurements
- TTL configuré par variables d'environnement
- Pub/Sub sur smart-campus:measurements, smart-campus:rooms, smart-campus:sensors, smart-campus:events
- Rate limiting partagé entre instances FastAPI via Redis

- Nginx distribue les requêtes entre backend-1, backend-2 et backend-3

5.5 Endpoints principaux

Endpoint	Usage	Protection
GET /health	Health simple	Public
GET /health/ready	MongoDB, InfluxDB, Redis	Public
POST /auth/register	Inscription utilisateur	Public
POST /auth/login	Connexion JWT	Public
GET /auth/me	Utilisateur courant	Bearer JWT
GET /rooms	Liste des salles	Public
POST /rooms	Création salle	Admin
GET /sensors	Liste capteurs	Public
POST /sensors	Création capteur	Admin
POST /measurements	Écriture mesure manuelle	Admin
GET /measurements/latest	Dernières mesures	Public (conception)
GET /measurements/history	Historique filtré	Public
POST /ingestion/measurements	Ingestion Node-RED/MQTT	X-API-Key
GET /metrics	Métriques Prometheus	Public

Note de sécurité : le test curl réalisé dans la partie Sécurité (section 4.2, A4) sur GET /measurements/latest sans jeton JWT a bien confirmé ce comportement — l'accès public à cette route est un choix de conception assumé par l'équipe backend, et non une faille non identifiée. Il reste néanmoins à évaluer collectivement si ce niveau d'exposition est acceptable pour la version finale du projet.

5.6 Variables d'environnement

MONGO_URI, MONGO_DATABASE · INFLUX_URL, INFLUX_TOKEN, INFLUX_ORG, INFLUX_BUCKET, INFLUX_INIT_PASSWORD · JWT_SECRET, JWT_ALGORITHM, JWT_ACCESS_TOKEN_EXPIRE_MINUTES · REDIS_URL, REDIS_SENTINEL_HOSTS, REDIS_SENTINEL_SERVICE, REDIS_DB · CACHE_TTL_SECONDS, LATEST_MEASUREMENTS_CACHE_TTL_SECONDS · RATE_LIMIT_ENABLED, RATE_LIMIT_REQUESTS, RATE_LIMIT_WINDOW_SECONDS · NODE_RED_API_KEY · INITIAL_ADMIN_EMAIL, INITIAL_ADMIN_PASSWORD · APP_HOST, APP_PORT, LOG_LEVEL, INSTANCE_ID

Les secrets sont lus depuis l'environnement. Un fichier backend/.env.example sert de modèle, tandis que .env est ignoré par Git.

5.7 Docker et exploitation

Service	Description
nginx	Reverse proxy et load balancer
backend-1/2/3	Instances FastAPI partageant les mêmes dépendances
mongodb	Persistance documentaire
influxdb	Persistance temporelle des mesures

Service	Description
redis-master	Redis primaire
redis-replica-1/2	Replicas Redis
redis-sentinel-1/2/3	Découverte du master et failover Redis

Commandes principales : `cd backend · copy .env.example .env · docker compose up -d --build · docker compose ps · python -m pytest --cov=app --cov-report=term-missing`

5.8 Tests et validation

- Dernière validation complète connue : 30 tests passés
- Couverture mesurée : 82 %, supérieure à l'objectif minimal de 70 %
- Validation Docker : MongoDB, InfluxDB, Redis, Sentinel, 3 backends et Nginx ont démarré sainement lors de la validation
- Validation HTTP : health, ready, auth admin, rooms, sensors, ingestion, latest/history, metrics et distribution Nginx ont été vérifiés
- Point restant : test long de failover Sentinel à refaire manuellement ; le premier essai a rencontré le mode TILT Sentinel et Redis était down pendant la fenêtre courte

5.9 Travail Git réalisé

Branche de travail : `feature/bdgl1-backend` · Commit poussé : `a238af4` — Finalize BD-GL 1 backend module · Remote : <https://github.com/ANGOROXANE/smart-campus-upb.git>

5.10 Limites et prochaines actions

- Refaire un test Sentinel avec attente longue après sortie du mode TILT
- Valider avec l'équipe RIST le format définitif Node-RED/MQTT
- Valider avec BD-GL 2 les endpoints de consommation dashboard
- Ajouter éventuellement des scripts de seed/dev pour simplifier les démonstrations

6. Partie Frontend & Observabilité

6.1 Présentation

Cette partie du projet concerne le développement du frontend, c'est-à-dire la réalisation des interfaces permettant aux utilisateurs de consulter et de superviser les informations du campus. L'objectif est de proposer une interface claire, responsive et exploitable depuis un navigateur.

Le frontend a été développé séparément du backend. Les interfaces ont d'abord été construites avec des données de démonstration, puis connectées progressivement aux endpoints REST fournis par le backend FastAPI.

6.2 Technologies utilisées

Technologie	Utilisation
React	Développement des composants et des pages de l'application
Vite	Environnement de développement et serveur frontend
Tailwind CSS	Mise en forme des interfaces et création d'une interface responsive
Recharts	Création du graphique d'évolution de la température
JavaScript	Logique frontend et appels à l'API
Fetch API	Communication entre le frontend et le backend REST

6.3 Architecture du frontend

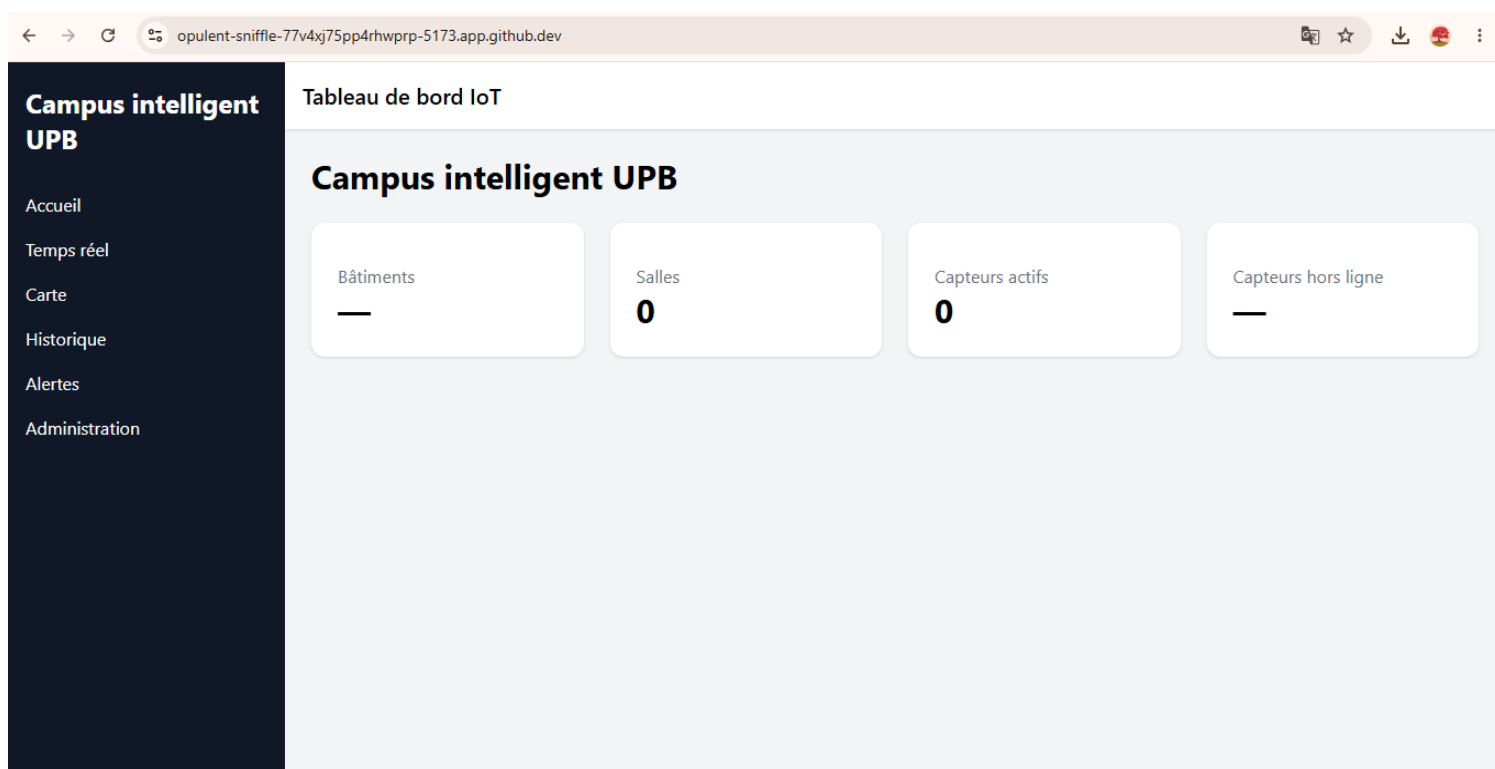
Le frontend est organisé autour de pages React et de services dédiés aux appels API. Un proxy Vite permet de rediriger les requêtes commençant par /api vers le backend FastAPI exécuté sur le port 8000.

Schéma logique :

Navigateur / utilisateur → Application React → Services API (api.js, measurements.js, sensors.js) → Proxy Vite /api → Backend FastAPI (port 8000) → MongoDB / InfluxDB / autres services backend

6.4.1 Accueil / Dashboard

La page d'accueil présente une synthèse du campus. Elle affiche notamment le nombre de salles et de capteurs à partir des données récupérées via l'API. Le nombre de bâtiments est calculé à partir des bâtiments associés aux salles.



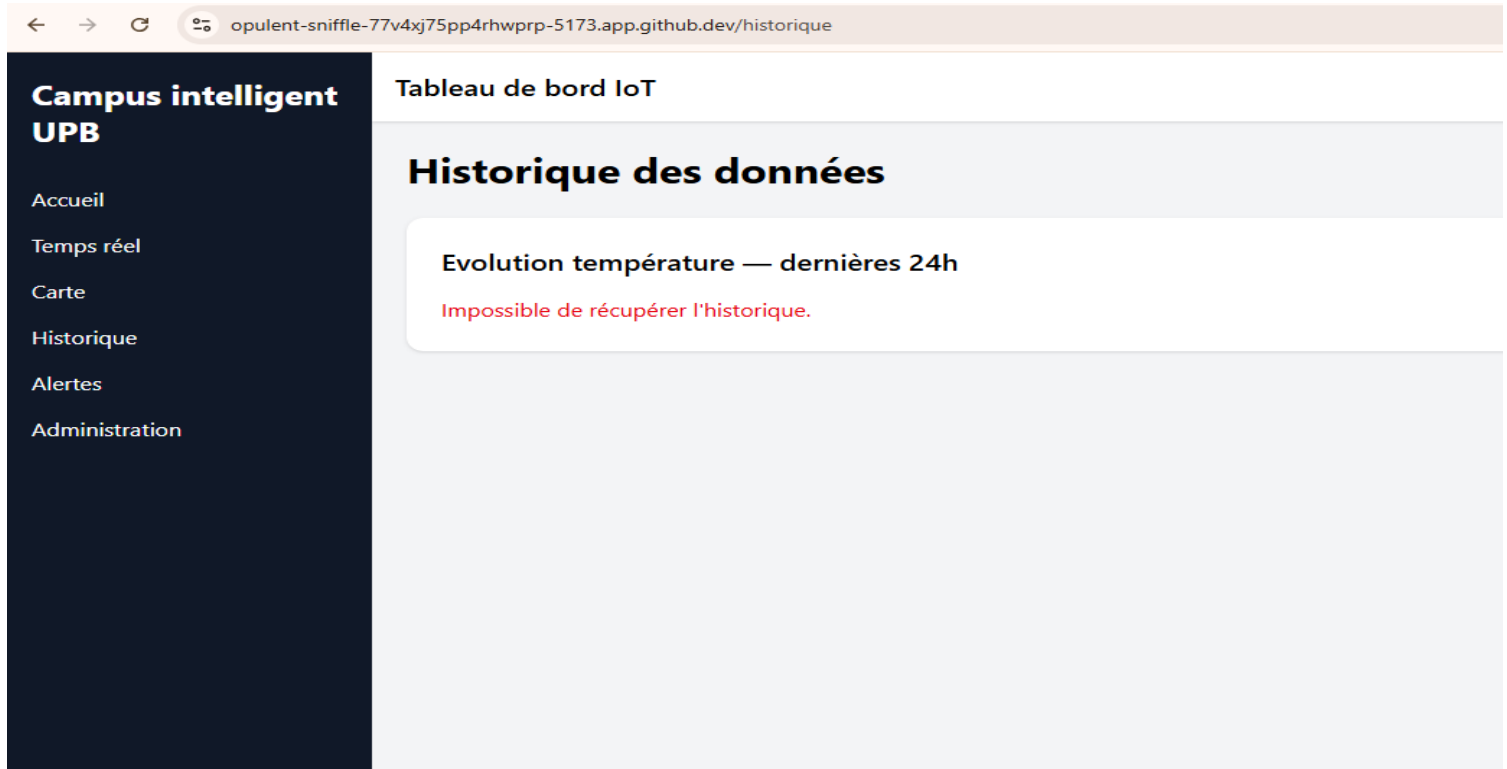
6.4.2 Temps réel

La page Temps réel affiche les dernières mesures disponibles. Elle utilise l'endpoint GET `/measurements/latest` et permet de présenter les informations issues des capteurs. Une actualisation périodique permet de maintenir les données visibles à jour.



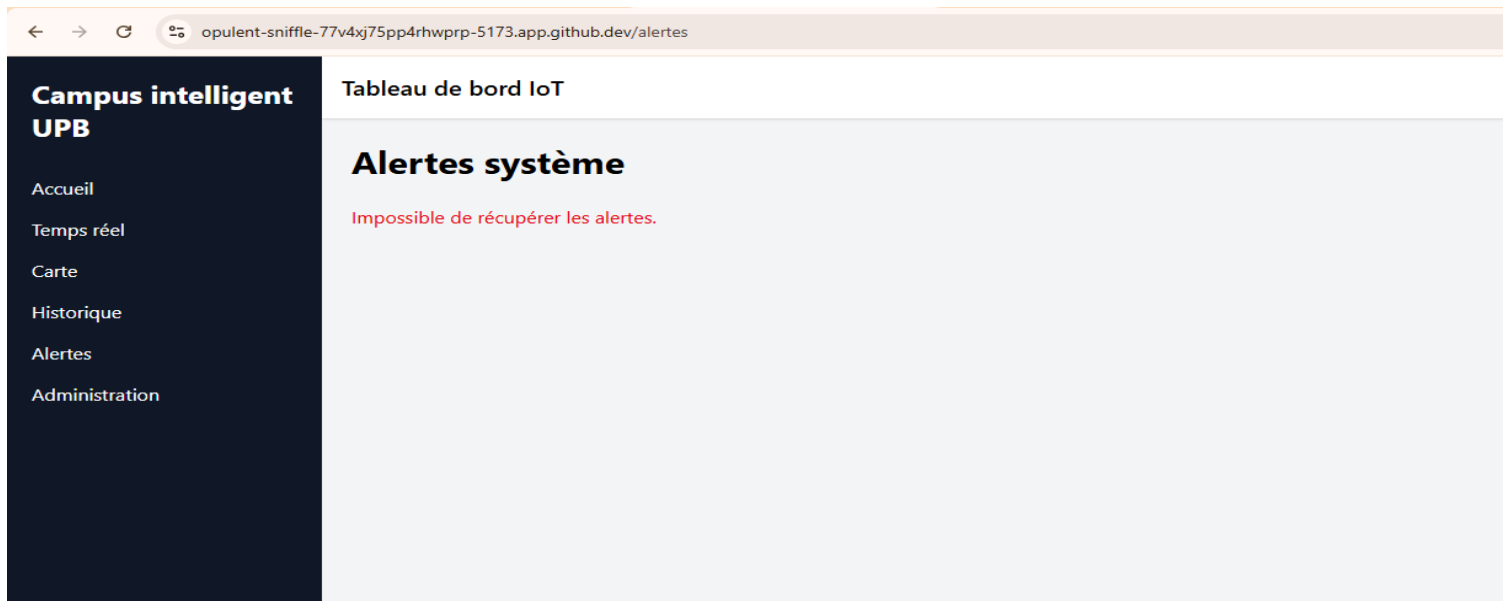
6.4.3 Historique —

La page Historique récupère les mesures des dernières 24 heures via GET /measurements/history. Les températures sont transformées en données adaptées à Recharts afin d'afficher une courbe d'évolution dans le temps.



6.4.4 Alertes —

La page Alertes exploite les dernières mesures pour générer des alertes côté frontend. Par exemple, une température supérieure ou égale à 30°C produit une alerte de niveau Attention. Les informations de présence peuvent également être signalées.



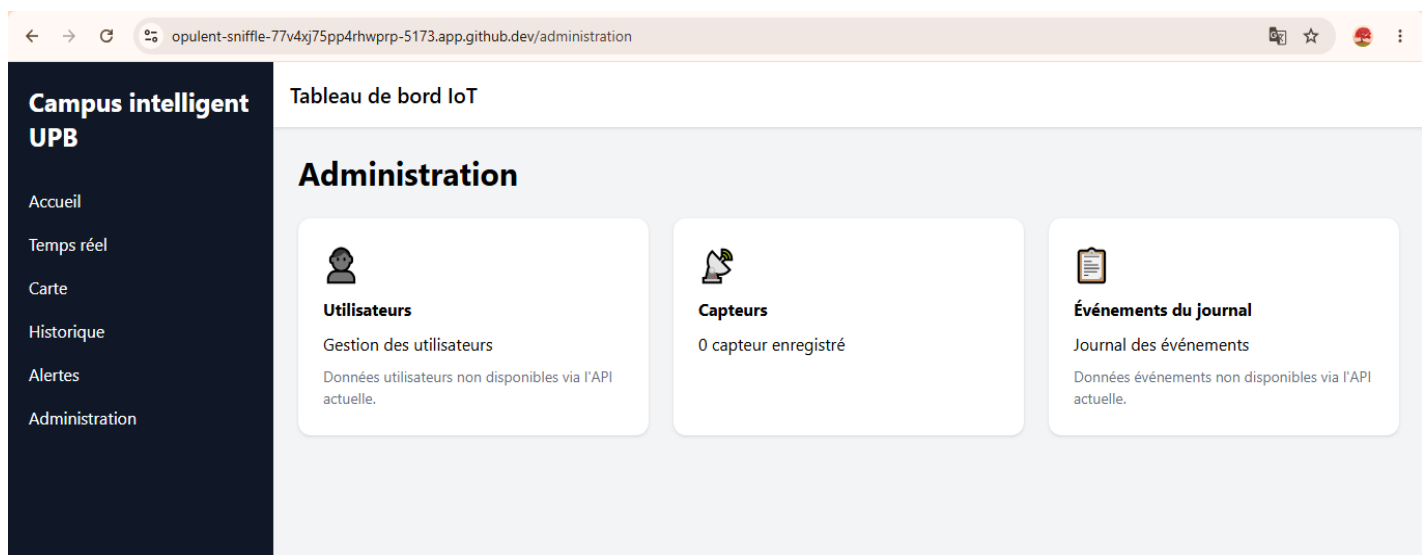
6.4.5 Carte du campus

La page Carte récupère les salles via GET /rooms et regroupe les salles par bâtiment. Elle permet ainsi d'obtenir une représentation simple du campus à partir des données réelles du backend, sans conserver des nombres de salles codés en dur.



6.4.6 Administration

— La page Administration présente les informations disponibles pour la gestion du campus. Le nombre de capteurs est récupéré dynamiquement via GET /sensors. Les données utilisateurs et le journal complet des événements n'étant pas exposés par les endpoints de lecture utilisés dans cette intégration, ils ne sont pas artificiellement inventés dans le frontend.



6.5 Tests et validation

La validation du frontend a été réalisée en lançant le serveur Vite et en vérifiant successivement les différentes pages de l'application :

- Vérification du démarrage du frontend avec `npm run dev`
- Vérification de l'accès au backend sur le port 8000
- Test de GET `/api/rooms`
- Test de récupération des capteurs
- Test des dernières mesures
- Test de l'historique des mesures
- Vérification des états vides et des messages d'erreur
- Vérification visuelle des pages Accueil, Temps réel, Historique, Alertes, Carte et Administration

6.6 Versionnement et travail collaboratif

Le travail frontend a été réalisé dans un GitHub Codespace sur le dépôt du projet. Les modifications ont ensuite été versionnées avec Git et poussées sur la branche main. Le commit final d'intégration du frontend porte le message « Connect frontend to backend API », contenant les pages React modifiées, la configuration du proxy Vite et les services d'accès à l'API.

6.7 Conclusion de la partie frontend

La partie frontend du projet Smart Campus UPB a permis de mettre en place une interface de supervision composée de six pages : accueil, temps réel, historique, alertes, carte du campus et administration. Les principales pages ont été connectées aux endpoints REST du backend afin de remplacer progressivement les données fictives par des données réelles. Le frontend est ainsi prêt à évoluer avec les données produites par le backend et les autres composants du système.

7. Conclusion générale

Sur le plan de la sécurité, les quatre scénarios d'attaque exigés (sniffing, MITM, replay, attaque API) ont été menés avec succès et documentés avec preuves à l'appui. La sécurisation TLS du canal MQTT a été mise en œuvre et validée expérimentalement, confirmant la disparition de la vulnérabilité de sniffing initialement démontrée. L'analyse STRIDE a permis de cartographier les six catégories de menaces du système et de proposer des contre-mesures concrètes, dont certaines restent à implémenter (rate limiting renforcé sur l'authentification, authentification systématique des routes de données).

Sur le plan du backend, une architecture robuste a été livrée : API FastAPI dockerisée avec authentification JWT et gestion des rôles, persistance combinant MongoDB (données métier) et InfluxDB (séries temporelles), cache Redis en haute disponibilité (Sentinel), et trois instances applicatives derrière un load balancer Nginx. Les tests automatisés couvrent 82 % du code, au-delà de l'objectif fixé à 70 %.

Sur le plan du frontend, six vues fonctionnelles ont été développées avec React et connectées progressivement aux données réelles de l'API (accueil, temps réel, historique, alertes, carte du campus, administration), remplaçant les données de démonstration initiales.

Limites du projet et perspectives d'amélioration :

- Le test de failover Sentinel en conditions longues n'a pas pu être validé intégralement (rencontre du mode TILT lors du premier essai) et reste à refaire
- L'authentification JWT n'est pas encore appliquée de manière systématique à l'ensemble des routes exposant des données ; ce choix de conception mériterait d'être réévalué collectivement
- Le format définitif d'ingestion Node-RED/MQTT reste à valider conjointement entre les trois membres de l'équipe
- Une authentification mutuelle par certificat (mutual TLS) côté capteurs permettrait de fermer la vulnérabilité de spoofing identifiée dans l'analyse STRIDE

Ce travail illustre concrètement les enjeux d'un système IoT réel ancré dans un contexte universitaire ivoirien : la nécessité de concilier disponibilité, performance et sécurité dès la phase de conception, plutôt que de traiter la sécurité comme une étape ajoutée après coup. La démarche attaque → preuve → contre-mesure → validation suivie tout au long du projet constitue une méthodologie transposable à d'autres systèmes IoT critiques (santé, énergie, transport) dans le contexte ouest-africain.